Cheat Sheet

Input & Output

1. print ("Hello", "World")
   output : Hello World

2. print ("=" * 30)
   output : print "=" 30 times

3. name = value
   "=" sign assigns a value to a variable.

4. type (x)
   Returns the type of a value
   str (text) int (whole number) float (decimal number)

5. Input ("...")
   It always return text string

6. int (x) : Convert to an integer
   float (x) : Converts to a decimal

7. f"{x:.2f}" → displays number with 2 decimal places
   f"{x:>10}" → right aligns the value in a field with 10 character wide.

137 // 60 → 2 (Quotient)
137 % 60 → 17 (Remainder)

(b) > Input () always give you the input data from the user and it is important to collect data
> "20" is the data stored in a string compared to 20 which is just a number
% is the modulo operator which is used to find the remainder after dividing one number by another. It can be used for algorithm and mathematics tasks.

01

1. **Comparisons**
   == [Equals to, which gives a true statement]
   != [Not equal to, gives a true statement]
   < [is less than, gives a false statement]
   > [is greater than, gives a true statement]
   <= [is less or equal to, a false statement]
   >= [is greater or equal to, a true statement]

== → is the equality operator / ask question
= → is assigement operator to store a value in a variable / store data

2. **Combining Conditions**
   and [logical AND, both statement has to be true to get true statement]
   or [logical OR, at least one statement has to be true to get true statement]
   not [logical NOT, statement is false]

3. **Decisions**
   if [if statement, Booleans decide whether the statement is true]
   elif [checked top to bottom. The first True statement runs and the rest skipped]
   else [It runs when the 'if condition' was False]

4. **Indentation**
   Intended lines are the block which needs to be respected in python. In loops for examples while using if statement.

5. **While loops**
   It repeats the block while condition is True
   It stops when condition becomes false (count ± 1)
   while True : + break → it loops until something inside the loop decides to stop instead of counting in advance.
   break and continue → the break exits the loop and continue skips the rest of the pass and goes straight to checking the condition

6. **For loops**
   range (stop) → when we know how many times to repeat
   range (3) → starts from 0 to 3
   range (start, stop, step) → Counting by more than one at a time, counting with a negative step
   Enumerate ( ) → It hands the position and the value together

7. **For or while**
   know the count in advance → For
   waiting on a condition or a signal → while

02

1. **def name (parameters)**
   defining creates a function, calling runs it. print ()
   Ex def greet ():
   print ("record check")

2. **return v/s print return**
   return : gives something back to use
   print () : Shows the output

3. **Argument : by position, by keyword & default value**
   position → values are passed in the same order as parameters are defined in the function (1st value → 1st parameter, 2nd value → 2nd parameter)
   keyword → pass values by parameter names while calling the function (order does not matter, more readable, less error prone)

4. ***args and **kwargs : any number of values or of named value**
   *args : collects any extra positional arguments into a tuple
   **kwargs : collects extra keyword arguments into a dict

5. **Scope**
   - A function's own variable do not leak out.
   - A variable created inside a function only exists inside it (Up local : local parameters)
   - return is not limited to 1 value, Separate them with commas
     Ex return difference, percent
   Function : Everything needs to come in a parameter, and what is provided goes out via return

6. **Docstrings**
   - Used to document python, provide a description of what a module, class, function or method does
   - declared in triple quotes (''' or """)

7. **Modules**
   - Import brings in a whole new module Ex (print (math.pi))
   - Accesses what is inside it with a dot

8. **if __name__ == "__main__"**
   - only run this part when this file is launched directly

03